```
houseRoast = null

hasMilk = (style) ->
  switch style
    when "latte", "cappuccino"
      yes
    else
      no

makeCoffee = (requestedStyle) ->
  style = requestedStyle || 'Espresso'
  if houseRoast?
    "#{houseRoast} #{style}"
  else
    style

barista = (style) ->
  time = (new Date()).getHours()
  if hasMilk(style) and time > 12 then "No!"
  else
    coffee = makeCoffee style
    "Enjoy your #{coffee}!"
```

A function to determine if a style of coffee has milk

A function to make the coffee; returns a string

A function that coffee is requested from

You don't yet fully grasp CoffeeScript, but already you can get a feel for the basic structure. An equivalent JavaScript program has a very similar structure but slightly different syntax, as you'll see in listing 2.2. Compare these programs side by side and you'll begin to understand the syntax features that CoffeeScript removes and appreciate *why* it removes them.

Why remove?

Claude Debussy is quoted as saying that "music is the space between the notes." CoffeeScript syntax is defined as much by what is missing as by what is present. Part of the thought experiment behind CoffeeScript is to take patterns written frequently in JavaScript, look at them, and ask, "How much of this is necessary?"

2.1.1. Comparing CoffeeScript to JavaScript